Modelling the 1-DOF Reaction-Wheel String Testbed

A step-by-step MATLAB guide

Calgary to Space — CTS-SAT-2
Attitude Determination and Control

October 20, 2026

Abstract

This guide builds, from first principles, a MATLAB model of a single-axis attitude testbed: a platform hanging on a string, carrying one reaction wheel and one IMU. It covers the equations of motion, how to measure every parameter on the bench, controller and estimator design, and the realistic discrete simulation loop that mirrors what the flight software will actually run.

Every number quoted was produced by running the code in this document. The parameter set is a representative 1U-class rig and must be replaced with your own measurements before any gain here is trusted.

The torsion string is treated as part of the plant, not as an inconvenience. It gives you one thing and takes another. Section 9 proves that the string makes platform angle observable from the gyro alone — and then Section 11 shows that the resulting observability is far too weak to use, which is the single most important design conclusion in this document. Section 14 prices the thing it takes.

Contents

1 What the testbed is, and what it is not **2**

2 Coordinates, assumptions and notation **3**

3 Equations of motion **3**

4 Actuator and sensor models **4**
4.1 Motor 4
4.2 IMU 5

5 Measuring the parameters on the bench **5**

6 Step 1 — Parameters **5**

7 Step 2 — The nonlinear plant, and the one check that matters **6**

8 Step 3 — Linear model and open-loop behaviour **7**

9 Step 4 — Controllability and observability **7**

10 Step 5 — Control design **8**
10.1 Two modes, not one 8
10.2 Sizing K_p and K_d 8
10.3 Why PD is not enough: add the integral term 9

11 Step 6 — The estimator, and the result that changes the rig **10**
11.1 Get Q from the physics, not from four tuned numbers . . . 11
11.2 The result . . . 11

12 Step 7 — Discretising and the realistic loop **12**

13 Step 8 — The same model in Simulink **13**
13.1 Build it in code; do not draw it . . . 13
13.2 Structure . . . 13
13.3 Verify it against the script . . . 13
13.4 Which one should you actually use? . . . 14

14 The string tax **14**
14.1 Momentum budget — check this before you build . . . 14
14.2 Holding an angle costs wheel speed . . . 15
14.3 The bearing-drag pointing bias . . . 15

15 Step 9 — The test campaign **16**

16 What this rig will not tell you about orbit **17**

A File manifest **18**

B Toolboxes, and the no-toolbox path **18**

C Reproducibility note **18**

---

1 What the testbed is, and what it is not

The rig is a mock satellite suspended from a thin filament so that it is free to rotate about the vertical axis and effectively locked in the other five degrees of freedom. A reaction wheel on the platform spins about that same vertical axis; an IMU on the platform measures the rate.

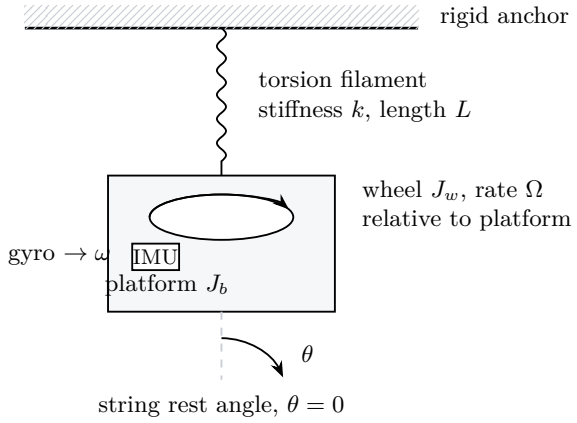


Figure 1: The single-axis rig. One rotational degree of freedom, θ, about the vertical. The filament supplies a restoring torque $-k\theta$ that orbit does not.

What it reproduces faithfully. Momentum exchange. The wheel is the only actuator, it can only trade momentum with the platform, and it saturates. Every control law you write has to live inside that constraint, and that is the lesson the testbed exists to teach. Sensor noise,

quantisation, loop delay, torque limits and bearing friction are all real on the bench and all behave the way they will in flight.

What it adds that orbit does not. The filament's torsional spring $-k\theta$ and the air drag $-b\omega$. Neither exists in space. They are not noise: they change the plant's pole structure, they make angle weakly observable from a rate sensor, and they force a standing wheel speed whenever you hold a non-zero angle.

What it cannot show you. Cross-axis gyroscopic coupling (the term $\boldsymbol{\omega} \times \mathbf{J}\boldsymbol{\omega}$ vanishes in one axis), nutation, gravity-gradient torque, and any three-axis determination question. The string also permits pendulum swing and conical motion; the model below assumes you have damped those out before taking data, and Section 15 says how to check.

2 Coordinates, assumptions and notation

Symbol	Meaning	Unit
θ	platform angle from the string's untwisted rest position	rad
ω	platform rate, $\omega = \dot{\theta}$, inertial	rad/s
Ω	wheel rate *relative to the platform* (what the encoder reads)	rad/s
J_b	inertia of everything *except* the rotor, about the vertical	$\mathrm{kg\,m^2}$
J_w	rotor inertia about its spin axis	$\mathrm{kg\,m^2}$
J_t	$J_b + J_w$, total system inertia (wheel locked)	$\mathrm{kg\,m^2}$
k	torsional stiffness of the suspension	N m/rad
b	viscous damping on the platform (air + filament)	N m s/rad
b_w	wheel bearing viscous drag	N m s/rad
τ_m	motor torque applied *to the rotor*	N m
τ_d	external disturbance torque on the platform	N m
h	total angular momentum about the vertical	N m s
β	gyro bias	rad/s

Table 1: Symbols. Note carefully that J_b *excludes* the rotor and that Ω is measured *relative to the platform*. Mixing those two conventions is the most common way to get this model wrong.

Assumptions.

(A1) Motion is confined to rotation about the vertical; pendulum and translational modes are damped out or ignored.

(A2) The wheel spin axis is exactly aligned with the suspension axis. A misalignment α scales the useful control torque by $\cos\alpha$ and dumps $\sin\alpha$ of it into the swing modes.

(A3) The filament is linear-elastic over the angular range used, so the restoring torque is $-k\theta$ with constant k. Verify this; see Section 5.

(A4) Damping is viscous. Bearing friction is in reality mostly Coulomb; that is added as a nonlinearity in Section 12.

3 Equations of motion

Work with total angular momentum about the suspension axis. The rotor spins at $\omega + \Omega$ in inertial terms, so

$$h \;=\; J_b\,\omega \;+\; J_w(\omega + \Omega) \;=\; J_t\,\omega \;+\; J_w\,\Omega. \tag{1}$$

The only torques crossing the system boundary are the filament, the air and any disturbance, so $\dot{h}$ equals their sum:

$$J_t\,\dot{\omega} + J_w\,\dot{\Omega} \;=\; -k\theta \;-\; b\,\omega \;+\; \tau_d. \tag{2}$$

The rotor alone obeys

$$J_w\left(\dot{\omega} + \dot{\Omega}\right) \;=\; \tau_m - b_w\Omega, \tag{3}$$

where $-b_w\Omega$ is the bearing drag acting on the rotor. Eliminating $\dot{\Omega}$ between (2) and (3), and using $J_t = J_b + J_w$, gives the two equations the model is built on:

$$J_b\,\dot{\omega} \;=\; -k\theta \;-\; b\,\omega \;-\; \tau_m \;+\; b_w\Omega \;+\; \tau_d \tag{4}$$

$$J_w\,\dot{\Omega} \;=\; \tau_m \;-\; b_w\Omega \;-\; J_w\dot{\omega} \tag{5}$$

Read (4) out loud, because the signs are the whole model. The platform feels $-\tau_m$: the *reaction* to the torque the motor puts into the rotor. It also feels $+b_w\Omega$: the reaction to the bearing drag. That second term looks negligible and is not — it produces a steady pointing bias proportional to stored wheel speed, derived and verified in Section 14.

With $x = [\theta,\ \omega,\ \Omega]^{\mathsf{T}}$ and input $u = \tau_m$,

$$\dot{x} = Ax + Bu, \qquad A = \begin{bmatrix} 0 & 1 & 0 \\ -\dfrac{k}{J_b} & -\dfrac{b}{J_b} & \dfrac{b_w}{J_b} \\ \dfrac{k}{J_b} & \dfrac{b}{J_b} & -\left(\dfrac{b_w}{J_w} + \dfrac{b_w}{J_b}\right) \end{bmatrix}, \qquad B = \begin{bmatrix} 0 \\ -\dfrac{1}{J_b} \\ \dfrac{1}{J_w} + \dfrac{1}{J_b} \end{bmatrix}. \tag{6}$$

This model is already linear — there is no $\omega \times J\omega$ term in a single axis — so (6) is exact, not a linearisation about an operating point. Every nonlinearity in this system comes from hardware: saturation, quantisation, friction.

Open-loop modes. With $\tau_m = 0$ the platform is a torsional oscillator:

$$\omega_n = \sqrt{k/J_t}, \qquad \zeta = \frac{b}{2\sqrt{kJ_t}}, \tag{7}$$

plus a slow real pole from the wheel coasting down through b_w.

4 Actuator and sensor models

4.1 Motor

Run the motor in *current control* if you possibly can. The inner current loop then makes torque the command, and $\tau_m = K_t i$ is linear and fast. Model the closed current loop as a first-order lag with limits:

$$\dot{\tau}_m = \frac{1}{\tau_c}\Big(\,\mathrm{sat}(\tau_{\mathrm{cmd}}, \pm\tau_{\max}) - \tau_m\Big). \tag{8}$$

Voltage control instead gives $\tau_m = \frac{K_t}{R}(V - K_e(\omega + \Omega))$, where the back-EMF term makes available torque fall off as the wheel speeds up — a real limit at high wheel speed that is often missed in student models.

Two hard limits bound everything the controller can do:

$$|\tau_m| \le \tau_{\max} \qquad \text{and} \qquad |h_w| = J_w|\Omega| \le J_w\Omega_{\max}. \tag{9}$$

The second is the momentum budget; Section 14 checks it.

4.2 IMU

The rate gyro gives

$$y_g[n] = \omega(t_n) + \beta[n] + v[n], \qquad \beta[n+1] = \beta[n] + w[n], \tag{10}$$

with white noise $v \sim \mathcal{N}(0, \sigma_v^2)$ where $\sigma_v = N/\sqrt{T_s}$ for an angle-random-walk density N, and a bias that random-walks under $w \sim \mathcal{N}(0, \sigma_w^2)$.

A decent consumer MEMS part has $N \approx 0.004\,^\circ/\mathrm{s}/\sqrt{\mathrm{Hz}}$, so at 100 Hz, $\sigma_v = 0.0400\,^\circ/\mathrm{s}$. A residual bias of $0.003\,^\circ/\mathrm{s}$ integrates to roughly 0.2° of heading error per minute. Pure gyro integration is therefore good for minutes, not hours — which is why Section 11 estimates β as a state instead of trusting it.

Pick the smallest full-scale range that covers your rates. A $\pm 2000\,^\circ/\mathrm{s}$ gyro quantises at $0.061\,^\circ/\mathrm{s}$ per LSB; the $\pm 250\,^\circ/\mathrm{s}$ range on the same part gives $0.0076\,^\circ/\mathrm{s}$, a factor of eight finer. Since this rig never exceeds about $30\,^\circ/\mathrm{s}$, use the narrow range. Also model the one-sample transport delay from the sampling instant to the torque actually changing.

5 Measuring the parameters on the bench

Do not take k from a formula. The textbook value for a solid circular shaft, $k = \pi G d^4/(32L)$, is wrong by an order of magnitude for braided line, and the fourth power on d means a 10 % diameter error is a 46 % stiffness error. Measure it.

Param.	Bench procedure	Reduction
J_t	Trifilar pendulum, or add a calibration mass of known inertia and re-run the torsion test.	$J_t = \dfrac{mgr^2T^2}{4\pi^2 L}$ for a trifilar rig of radius r, length L.
J_w	Weigh the rotor and measure its radii; confirm by spin-down.	$J_w = \frac{1}{2}mr^2$ for a uniform disc.
k	Lock the wheel, twist to 20°, release, log the gyro for ten full cycles.	$k = J_t \left(\dfrac{2\pi}{T_{\mathrm{osc}}}\right)^2$
b	Same run — fit the decay envelope of the gyro trace.	$\delta = \frac{1}{n}\ln\frac{A_0}{A_n}$, $\zeta = \delta/\sqrt{4\pi^2+\delta^2}$, $b = 2\zeta\sqrt{kJ_t}$
b_w	Clamp the platform, spin the wheel up, cut drive, log the encoder coast-down.	Fit $\Omega(t) = \Omega_0 e^{-t/\tau_w}$, then $b_w = J_w/\tau_w$.
K_t	Command known currents against a torque arm on a scale.	Slope of τ_m against i.

Table 2: How to get each number. The torsion-release test gives both k and b from one run and is the single most important measurement on the rig.

Check linearity while you are there. Repeat the release test from 5°, 20° and 60°. If the measured period T_{osc} drifts by more than a few percent across that range then assumption (A3) is broken, the filament is not linear-elastic over your working range, and you must either shorten the angular range or fit $k(\theta)$. Monofilament also creeps: hang the rig overnight under load before trusting any of these numbers, and re-zero θ at the start of every session because the untwisted rest angle wanders.

6 Step 1 — Parameters

Keep every number in one file, in SI, with the measurement that produced it written beside it. Nothing else in the project should contain a literal physical constant.

```
function p = rw1dof_params()
%RW1DOF_PARAMS  Measured parameters, 1-DOF string testbed. All SI.
%   Replace every value with your own measurement before trusting a gain.
p.Jb   = 2.15e-3;   % kg m^2  platform + stator + payload, rotor EXCLUDED
p.Jw   = 1.08e-5;   % kg m^2  wheel rotor about spin axis (disc, weighed)
p.Jt   = p.Jb + p.Jw;
p.k    = 4.80e-5;   % N m/rad   from T_osc = 42.2 s, wheel locked
p.b    = 6.40e-6;   % N m s/rad from log decrement, zeta = 0.0099
p.bw   = 1.10e-7;   % N m s/rad from wheel coast-down, tau_w = 98 s
p.tc   = 8.0e-6;    % N m       Coulomb bearing torque (see Sec. 12)
p.tauc = 0.015;     % s         closed current-loop time constant
p.tmax = 2.00e-3;   % N m       motor torque limit
p.wmax = 838;       % rad/s     wheel speed limit (8000 rpm)
p.Ts   = 0.01;      % s         IMU sample period (100 Hz)
end
```

Listing 1: `rw1dof_params.m`

7 Step 2 — The nonlinear plant, and the one check that matters

Equations (4) and (5) transcribe directly.

```
function dx = rw1dof_ode(~, x, tau_m, p, tau_d)
%RW1DOF_ODE  xdot for x = [theta; omega; Omega_rel].
th = x(1); w = x(2); Om = x(3);
dw  = (-p.k*th - p.b*w - tau_m + p.bw*Om + tau_d) / p.Jb;
dOm = (tau_m - p.bw*Om)/p.Jw - dw;
dx  = [w; dw; dOm];
end
```

Listing 2: `rw1dof_ode.m`

Before any control design, prove the coupling signs are right. Switch off every external torque ($k = b = b_w = 0$). The system is then isolated, so the total momentum (1) must be conserved no matter what the motor does. This one assertion catches every sign error in (4) and (5), which is the only class of bug that quietly produces a plausible-looking but backwards simulation.

```
p = rw1dof_params();
q = p; q.k = 0; q.b = 0; q.bw = 0;          % isolate the system
tau = @(t) 1e-3*sin(2*pi*0.3*t);            % arbitrary motor torque
opt = odeset('RelTol',1e-10,'AbsTol',1e-12);
[~,X] = ode45(@(t,x) rw1dof_ode(t,x,tau(t),q,0), [0 20], [0;0.2;0], opt);

h = q.Jt*X(:,2) + q.Jw*X(:,3);              % Eq. (1)
drift = max(abs(h - h(1)))/abs(h(1));
fprintf('momentum drift %.2e (h0 = %.4e N m s)\n', drift, h(1));
assert(drift < 1e-9, 'Plant violates momentum conservation: check your signs.')
```

Listing 3: Momentum self-check. Re-run this after any edit to the plant.

Running it returns a relative drift of 2.3×10^{-15} on $h_0 = 4.3216 \times 10^{-4}\,\mathrm{N\,m\,s}$, which is integrator round-off. The plant is right.

8 Step 3 — Linear model and open-loop behaviour

```
function [A,B] = rw1dof_lin(p)
A = [      0            1              0
      -p.k/p.Jb   -p.b/p.Jb      p.bw/p.Jb
       p.k/p.Jb    p.b/p.Jb  -(p.bw/p.Jw + p.bw/p.Jb) ];
B = [0; -1/p.Jb; 1/p.Jw + 1/p.Jb];
end
```

Listing 4: `rw1dof_lin.m`

```
p = rw1dof_params();  [A,B] = rw1dof_lin(p);
sys = ss(A,B,eye(3),0, 'StateName',{'theta','omega','Omega'}, ...
                       'InputName','tau_m');
damp(sys)
wn = sqrt(p.k/p.Jt);  zeta = p.b/(2*sqrt(p.k*p.Jt));
fprintf('torsional mode: wn %.4f rad/s (T = %.1f s), zeta %.4f\n', ...
        wn, 2*pi/wn, zeta);
```

Listing 5: Open-loop survey

For the parameters above this gives a torsional mode at $\omega_n = 0.1490\,\mathrm{rad/s}$ — a period of $42.2\,\mathrm{s}$ — with $\zeta = 0.0099$, and the full eigenvalues of A come out as $-0.0015 \pm 0.1494\mathrm{i}$ and -0.0102, the last being the wheel coasting down. The rig is a very lightly damped, very slow oscillator. Two consequences follow immediately:

1. **Open-loop tests take minutes.** A free-decay run needs $\sim 10\,T_{\mathrm{osc}} \approx 7\,\mathrm{min}$ of clean data. Budget for it, and make sure nothing bumps the bench.
2. **Your controller will dominate the string.** A closed-loop bandwidth of $1.2\,\mathrm{rad/s}$ sits a factor of eight above ω_n, so the spring is a small perturbation on the closed-loop *dynamics* — but a dominant one on the *wheel speed*, as Section 14 shows.

Always cross-check the linear model against the nonlinear one before believing either. They should overlay to integrator tolerance, since the plant is linear:

```
tau = @(t) 5e-4*(t>1);
[T1,X1] = ode45(@(t,x) rw1dof_ode(t,x,tau(t),p,0), [0 60], [0;0;0]);
Y2 = lsim(sys, arrayfun(tau,T1), T1);
fprintf('max |theta| mismatch: %.2e rad\n', max(abs(X1(:,1)-Y2(:,1))));
```

9 Step 4 — Controllability and observability

Controllability is unsurprising: `rank(ctrb(A,B))` returns 3. One wheel controls one axis.

Observability is where the string earns its keep. Take the practical case: the only sensor is a rate gyro, and its bias β is unknown, so augment the state to $[\theta,\ \omega,\ \beta]$ with measurement $y = \omega + \beta$. The question is whether θ and β can be separated at all.

They can, *provided* $k \neq 0$. Working out the observability matrix by hand with $C = [0\ 1\ 1]$:

$$\mathcal{O} = \begin{bmatrix} 0 & 1 & 1 \\ -k/J_b & -b/J_b & 0 \\ bk/J_b^2 & -k/J_b + b^2/J_b^2 & 0 \end{bmatrix}, \qquad \det \mathcal{O} = \frac{k^2}{J_b^2}. \tag{11}$$

The determinant depends on k and nothing else. With the string attached the triple is observable; cut the string and the matrix drops rank, because a constant gyro bias and a slowly turning platform become indistinguishable. The restoring torque is what breaks the tie: it makes the platform's *rate* depend on its *angle*, so angle information leaks into a rate measurement.

```
C  = [0 1 1];                          % y = omega + bias
Aa = [A(1:2,1:2) [0;0]; 0 0 0];        % [theta omega beta]
fprintf('rank (k = %.2e): %d   det = %.4e\n', ...
        p.k, rank(obsv(Aa,C)), det(obsv(Aa,C)));
A0 = Aa; A0(2,1) = 0;                  % same rig, string cut
fprintf('rank (k = 0)     : %d   det = %.4e\n', ...
        rank(obsv(A0,C)), det(obsv(A0,C)));
fprintf('predicted k^2/Jb^2 = %.4e\n', p.k^2/p.Jb^2);
```

Listing 6: Verifying Eq. (11) numerically

Output: rank 3 and det $= 4.9843 \times 10^{-4}$ with the string, rank 2 and det $= 0$ without, and the predicted $k^2/J_b^2 = 4.9843 \times 10^{-4}$ matches to all printed digits.

Do not celebrate yet. Observable is a yes/no property and it is answering the wrong question. What matters is *how strongly* observable, and $\det \mathcal{O} \propto k^2$ with a very small k is a warning, not a result. Section 11 builds the estimator this analysis invites and then measures how badly it performs.

10 Step 5 — Control design

The controller for this rig is a PID, and nothing more exotic. It is built in two stages: Section 10.2 sizes K_p and K_d from the closed-loop response you want, and Section 10.3 shows why the derivative-plus- proportional pair alone leaves a measurable pointing error and what the integral term costs to remove it. Every gain comes out of a closed-form expression in the measured parameters — there is no optimisation step and no state-space machinery to take on faith.

10.1 Two modes, not one

Real ADCS software does not run one controller. It runs at least two, and this rig needs the same split:

Mode 1, detumble. Kill the rate. Uses the gyro only, needs no angle knowledge, and is the mode that runs right after you let go of the platform.

Mode 2, point. Hold θ at a commanded value. Needs an angle estimate and is therefore only as good as Section 11.

Keeping them separate means a bad angle estimate cannot destabilise your detumble, which is the one thing that must always work.

10.2 Sizing K_p and K_d

Treat the current loop as fast and the wheel drag as negligible, so (4) reduces to $J_b\ddot{\theta} = -k\theta - b\dot{\theta} - \tau_m$. With $\tau_m = K_p\hat{\theta} + K_d\hat{\omega}$ the closed loop becomes

$$J_b\ddot{\theta} + (b + K_d)\dot{\theta} + (k + K_p)\theta = 0, \tag{12}$$

so for a target ω_c, ζ_c,

$$K_p = J_b\omega_c^2 - k, \qquad K_d = 2\zeta_c\omega_c J_b - b. \tag{13}$$

Note that the gains *subtract* the plant's own spring and damping: the string is doing part of your job, and the controller only supplies the difference.

```
wc = 1.2; zc = 0.7;                          % target closed loop
Kp = p.Jb*wc^2 - p.k;
Kd = 2*zc*wc*p.Jb - p.b;
fprintf('Kp = %.4e N m/rad   Kd = %.4e N m s/rad\n', Kp, Kd);

step_deg = 30;                               % largest step you intend to command
tau_pk = Kp*step_deg*pi/180;
verdict = 'OK';
if tau_pk >= p.tmax, verdict = 'SATURATES - lower wc'; end
fprintf('torque for %d deg step = %.4e N m (limit %.2e) -> %s\n', ...
    step_deg, tau_pk, p.tmax, verdict);
fprintf('settling time ~ %.1f s\n', 4/(zc*wc));
```

Listing 7: PD gains, with the saturation check that must accompany them

This returns $K_p = 3.048 \times 10^{-3}\,\mathrm{N\,m/rad}$, $K_d = 3.606 \times 10^{-3}\,\mathrm{N\,m\,s/rad}$, a peak demand of $1.596 \times 10^{-3}\,\mathrm{N\,m}$ against the $2.0 \times 10^{-3}\,\mathrm{N\,m}$ limit, and a settling time of 4.8 s. Raising ω_c to 1.5 rad/s would saturate on a 30° step — which is why the saturation line sits in the same code block as the gain line. A gain you have not checked against $\tau_{\max}$ is not a design.

10.3 Why PD is not enough: add the integral term

PD leaves a steady pointing error, and not by accident. Setting $\dot{\omega} = \omega = 0$ in (4) with $\tau_m = K_p\theta$ gives

$$\theta_{\mathrm{ss}} = \frac{f(\Omega)}{k + K_p}, \qquad f(\Omega) = b_w\Omega + \tau_c, \tag{14}$$

where $f(\Omega)$ is the bearing drag whose reaction appears in (4). A proportional controller *needs* a non-zero θ to produce the torque that balances that drag, so the error cannot go to zero. The simulated PD loop settles at 0.2565°, and (14) predicts the same figure — this is a structural property of the controller, not noise and not a tuning failure.

Removing a constant offset is exactly what integral action is for:

$$\tau_m = K_p\theta + K_d\omega + K_i\int \theta\,\mathrm{d}t. \tag{15}$$

How much K_i can you use? Adding the integrator makes the loop third order,

$$J_b s^3 + (b + K_d)s^2 + (k + K_p)s + K_i = 0, \tag{16}$$

and the Routh criterion gives a hard stability ceiling

$$K_i < \frac{(b + K_d)(k + K_p)}{J_b} = 5.20 \times 10^{-3}\,\mathrm{N\,m/(rad\,s)}. \tag{17}$$

That is a bound, not a target. Sweeping K_i through the nonlinear model:

But gate it, and clamp it. Two conditions come attached, and skipping either one makes the integrator worse than useless:

1. **Integral action belongs to pointing mode only.** During detumble θ is sweeping through large angles and its integral grows without meaning anything. Leaving the integrator running through a detumble from 30 °/s, with no anti-windup, produces 11.6° **of overshoot** when pointing mode takes over — far worse than the 0.26° of bias it was meant to fix.

K_i	% of Eq. (17)	final θ	wheel speed	$\lvert\theta\rvert$ after 60 s
0 (pure PD)	0 %	0.2565°	53.3 rad/s (6.4 % sat.)	0.271°
$K_p/10$	5.9 %	0.0000°	62.3 rad/s (7.4 % sat.)	0.091°
$\mathbf{K_p/5}$	11.7 %	0.0000°	62.4 rad/s (7.4 % sat.)	0.003°
$K_p/2$	29.3 %	0.0000°	62.5 rad/s (7.5 % sat.)	0.000°

Table 3: Integral action removes the bias outright. The objection you would expect — that a standing torque command spins the wheel up — is real but small here: the whole bias torque is only about 1.4×10^{-5} N m, so buying zero steady-state error costs about 9 rad/s of wheel speed, one percentage point of saturation. Take $K_i = K_p/5$.

2. **Clamp the integrator against the torque limit.** While $|\tau_{\text{cmd}}| \geq \tau_{\max}$ the extra torque the integrator asks for does not exist, so accumulating it only guarantees a long unwinding later. Stop integrating whenever the command is saturated *and* further integration would push it deeper into the limit.

```
Ki = Kp/5;                                  % Table 6; 11.7% of the Routh limit

if pointing
    u_raw = Kp*th_hat + Kd*w_hat + Ki*xi;
else
    u_raw = Kd*w_hat;                       % MODE 1: no integral action
    xi    = 0;                              % and hold the accumulator at zero
end
u = max(-p.tmax, min(p.tmax, u_raw));       % saturate

if pointing && (abs(u_raw) < p.tmax || sign(th_hat) ~= sign(u_raw))
    xi = xi + p.Ts*th_hat;                  % clamping anti-windup
end
```

Listing 8: PID with mode gating and clamping anti-windup

Where the integrator runs out of road. The wheel-speed cost above is small because the disturbance it fights is small. Commanding a *non-zero* θ_r is a different matter: the filament then demands a standing torque $k\theta_r$, which is far larger than $f(\Omega)$, and the integrator supplies it by spinning the wheel toward saturation at the rate derived in Section 14. Integral action does not repeal that; it just makes the controller pursue it faithfully. Hold $\theta_r = 0$ for long runs and treat offset holds as short manoeuvres.

11 Step 6 — The estimator, and the result that changes the rig

Read the conclusion even if you skip the machinery. This is the one chapter that uses state-space estimation, and the Kalman filter below is doing analysis, not control — the PID of Section 10.3 is still the whole controller. If the matrix algebra is ahead of where the team is, read Table 4 and the paragraph after it and move on: the finding is a *hardware* requirement, and it stands regardless of which filter you eventually implement. A fixed-gain complementary filter reaches the same conclusion; the Kalman form is used here only because it makes the convergence time constant something you can read off directly rather than tune for.

Augment the plant with the gyro bias, $x_a = [\theta,\ \omega,\ \Omega,\ \beta]$.

11.1 Get Q from the physics, not from four tuned numbers

The most common estimator mistake is to type four small numbers into `diag()` and tune them until something looks right. Do not. Ask instead what the model does not know, which is an unmodelled torque on the platform. That torque enters exactly where τ_d enters in (4), so it has a known direction in state space:

$$G = \begin{bmatrix} 0 & 0 \\ 1/J_b & 0 \\ -1/J_b & 0 \\ 0 & 1 \end{bmatrix}, \qquad Q_c = \mathrm{diag}\left(\sigma_\tau^2,\ \sigma_\beta^2\right), \qquad Q_d = G\,Q_c\,G^{\mathsf{T}}\,T_s. \tag{18}$$

Now there are two knobs with physical meaning — how large an unmodelled torque you expect (2×10^{-5} N m here, covering Coulomb friction, air currents and filament irregularity) and how fast the gyro bias wanders — instead of four numbers with none.

```
Aa = [A zeros(3,1); zeros(1,4)];   Ba = [B; 0];
Ca = [1 0 0 0;      % absolute angle   (see the result below)
      0 1 0 1;      % gyro, biased
      0 0 1 0];     % wheel encoder

sig_tau = 2e-5;              % N m,   1-sigma unmodelled torque
sig_brw = 2e-4*pi/180;       % rad/s per sqrt(s), bias random walk
Gw = [0 0; 1/p.Jb 0; -1/p.Jb 0; 0 1];

N   = 0.004*pi/180;          % gyro ARW density
sig = [0.5*pi/180, N/sqrt(p.Ts), 0.5];        % angle, gyro, encoder sigmas

d   = c2d(ss(Aa,Ba,Ca,0), p.Ts, 'zoh');
M   = dlqe(d.A, eye(4), d.C, Gw*diag([sig_tau^2 sig_brw^2])*Gw'*p.Ts, ...
           diag(sig.^2));
fprintf('slowest estimator tau = %.1f s\n', ...
        -p.Ts/log(max(abs(eig((eye(4)-M*d.C)*d.A)))));
```

Listing 9: Steady-state discrete estimator

(`dlqe` is legacy but still ships; `kalman` on the discrete model is the modern equivalent and gives the same gain.)

11.2 The result

Run the full closed loop twice — once with gyro and encoder only, as Section 9 suggests is sufficient, and once with an absolute angle measurement added at 0.5° noise. Identical plant, identical controller, identical noise seed:

The conclusion, and it is a hardware conclusion. Section 9 is correct and insufficient. Angle information reaches the gyro only through the k/J_b term, which for this rig is 0.0223/s^2; the resulting estimator needs 488 s to converge on a plant whose own period is 42 s. By the time the estimate is good the run is over. **Budget for an absolute angle sensor on the testbed.** A webcam watching a fiducial marker on the platform, a low-friction potentiometer at the anchor, or a magnetometer heading (only if the bench is free of ferrous clutter) all work; 0.5° of noise is enough, as case B shows. Detumble is unaffected either way, which is exactly why the two modes are kept separate.

This is the most important thing the model told us, and no amount of staring at rank($\mathcal{O}$) = 3 would have revealed it.

	A: gyro + encoder	B: + absolute angle
Slowest estimator time constant	488 s	98 s
Detumble, $\lvert\omega\rvert < 1\,^\circ$/s	33.1 s	33.1 s
Peak wheel speed	179 rad/s (21 % sat.)	100 rad/s (12 % sat.)
Pointing, mean θ over last 20 s	-23.06°	0.14°
Pointing, RMS θ	23.10°	0.14°
Bias estimate (true $0.0030\,^\circ$/s)	$-0.1832\,^\circ$/s	$-0.0005\,^\circ$/s

Table 4: Case A is observable on paper and useless in practice. The estimator cannot separate θ from β in any usable time, so it mis-estimates the bias by two orders of magnitude and the controller confidently parks the platform 23° away from where it was told to.

12 Step 7 — Discretising and the realistic loop

The continuous simulation proves the design. The discrete one proves the *implementation*, so build it to match the flight software exactly: sample, estimate, compute, apply, hold.

```
Ad = d.A; Bd = d.B; Cd = d.C;
Kp = p.Jb*1.2^2 - p.k;  Kd = 2*0.7*1.2*p.Jb - p.b;

f   = @(Om) p.bw*Om + p.tc*tanh(Om/1.0);     % viscous + Coulomb bearing drag
ode = @(x,tm) [ x(2);
   (-p.k*x(1) - p.b*x(2) - tm + f(x(3)))/p.Jb;
   (tm - f(x(3)))/p.Jw - (-p.k*x(1) - p.b*x(2) - tm + f(x(3)))/p.Jb ];

Ts = p.Ts;  n = round(180/Ts);
x  = [0; 30*pi/180; 0];                      % hand spin-up, 30 deg/s
tm = 0;  xh = zeros(4,1);  beta = 0.003*pi/180;
lsb = (1/131)*pi/180;                        % +-250 dps gyro LSB
rng(7)

for i = 1:n
    t  = i*Ts;
    yg = lsb*round((x(2) + beta + sig(2)*randn)/lsb);   % sample + quantise
    ye = x(3) + sig(3)*randn;
    ya = x(1) + sig(1)*randn;
    if i == 1, xh = [ya; yg; ye; 0]; end                % initialise, do not zero
    xh = xh + M*([ya; yg; ye] - Cd*xh);                 % correct

    if t < 30                                           % MODE 1 detumble
        u_raw = Kd*xh(2);  xi = 0;                      % integrator held down
    else                                                % MODE 2 point
        u_raw = Kp*xh(1) + Kd*xh(2) + Ki*xi;
    end
    u = max(-p.tmax, min(p.tmax, u_raw));               % saturate
    if t >= 30 && (abs(u_raw) < p.tmax || sign(xh(1)) ~= sign(u_raw))
        xi = xi + Ts*xh(1);                             % clamping anti-windup
    end

    for s = 1:10                                        % hold u for one Ts
        tm = tm + (Ts/10)/p.tauc*(u - tm);              % current-loop lag
        x  = x  + (Ts/10)*ode(x, tm);
    end
    xh = Ad*xh + Bd*tm;                                 % predict
end
```

Listing 10: The realistic closed loop. This is the structure the firmware mirrors.

Four details in that loop are the difference between a simulation that predicts the bench and one that flatters you:

1. **Quantisation before the estimator.** Rounding to the gyro LSB is what sets the noise floor at low rates, not the datasheet noise density.
2. **Coulomb friction.** `p.tc*tanh(Om/1.0)` dominates viscous drag near zero wheel speed and is the usual cause of a stubborn limit cycle on real hardware. The `tanh` is a smooth stand-in for sign so the integrator does not chatter; keep the width small relative to your working wheel speeds.
3. **The zero-order hold.** Torque is computed once per T_s and held. Integrating the plant with a fine sub-step inside that hold is what reveals inter-sample behaviour.
4. **Initialise the estimate from the first measurement.** Starting `xh` at zero when the platform is already at 30 °/s injects a huge transient that is not real.

13 Step 8 — The same model in Simulink

A block diagram is worth having for this rig: it shows the momentum exchange as a picture, it is the artefact most readable by someone who has not read the code, and it is the usual starting point for Embedded Coder if the controller is eventually generated rather than hand-written.

13.1 Build it in code; do not draw it

The model is produced by `rw1dof_build_slx.m`, which calls `add_block` and `add_line` to assemble it from nothing. That is more work than dragging blocks once, and it pays for itself immediately:

- **A `.slx` is a binary.** Committed as-is it cannot be diffed, cannot be reviewed in a pull request, and cannot be merged when two people edit it in the same week. The builder script is plain text, so all three work normally.
- **It cannot drift from the parameters.** The builder reads `rw1dof_params.m`, so there is no second copy of J_b to forget.
- **It is reproducible.** Delete the model, re-run one file, get the identical model back.

```
rw1dof_build_slx     % writes rw1dof.slx from rw1dof_params.m
rw1dof_verify_slx    % checks it against ode45, exports the diagrams
```

Listing 11: Building and checking the model

13.2 Structure

Three subsystems, nine blocks at the top level (Figure 2). Flat diagrams of a feedback loop this size are unreadable on a page, and `Simulink.BlockDiagram.arrangeSystem` does not rescue them — the builder places blocks deliberately and never calls it.

The controller subsystem is Figure 3 and the plant is Figure 4. Parameters live in the model workspace, written there by the builder, so the `.slx` runs standalone without anything in the base workspace.

13.3 Verify it against the script

A block diagram that runs has proved nothing. Two independent implementations of the same equations should agree, and if they do not, one is wrong. `rw1dof_verify_slx.m` runs both over 180 s and compares:

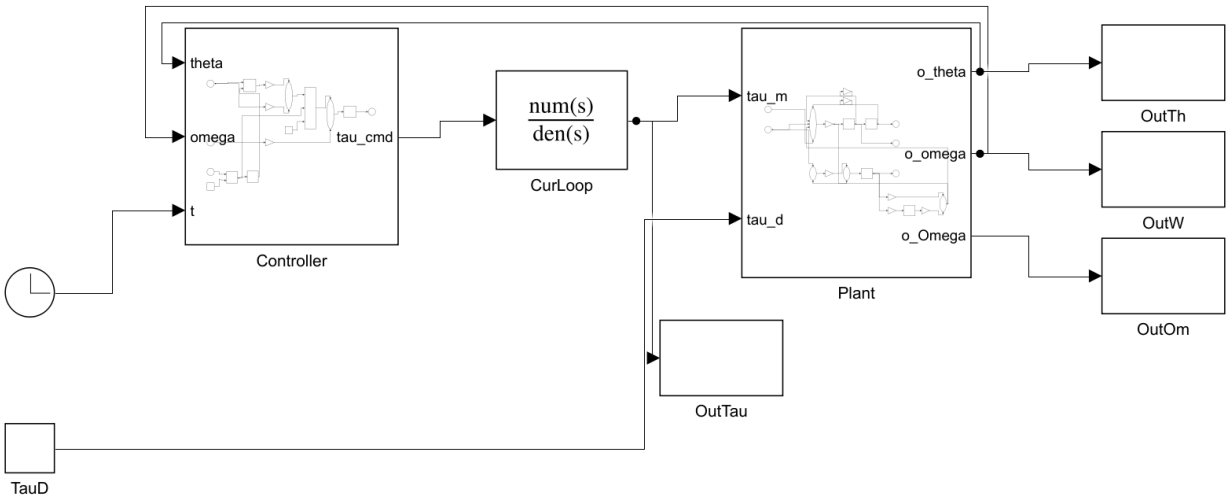


Figure 2: Top level. Controller → current-loop lag → plant, with θ and ω fed back. The disturbance input `tau_d` is wired and held at zero, ready for the disturbance test of Section 15.

Check	Result	Verdict
max $\lvert\Delta\theta\rvert$ vs `ode45`	1.18×10^{-4} rad (0.0068°)	pass
max $\lvert\Delta\omega\rvert$ vs `ode45`	2.04×10^{-4} rad/s	pass
max $\lvert\Delta\Omega\rvert$ vs `ode45`	4.1×10^{-2} rad/s (0.04 % of peak)	pass
Steady-state θ with integral action	0.000 00°	pass
Final wheel speed	62.41 rad/s	matches Table 3

Table 5: The residuals are two solvers disagreeing at their tolerances, not a modelling difference. The last row is the useful one: the block diagram lands on 62.4 rad/s, the same figure the $K_i = K_p/5$ row of Table 3 produced from a completely separate script.

Note what the steady-state row is really testing. Had the integrator gate, the reset polarity or the sign of the friction feedback been wrong, θ would have settled at the PD value of about 0.275° — or wandered off — and the trajectory comparison alone might still have looked plausible. Checking a quantity with a known closed form is a sharper test than checking that two curves overlay.

13.4 Which one should you actually use?

Keep both, for different jobs. The script model is what you iterate in: it runs headless, it diffs, it drops into CI, and it is what the STM32 firmware will look like. The Simulink model is what you show people, what you hand to Embedded Coder, and where adding a plant nonlinearity is a two-block change rather than an edit to a right-hand side. Because the builder regenerates the `.slx` from the same `rw1dof_params.m`, they cannot silently disagree about the hardware.

14 The string tax

14.1 Momentum budget — check this before you build

$$h_w^{\max} = J_w \Omega_{\max} \qquad \text{must exceed} \qquad h_{\text{plat}} = J_b\, \omega_{\max}. \tag{19}$$

Here $h_w^{\max} = 9.05 \times 10^{-3}$ N m s against 1.126×10^{-3} N m s for the platform at 30 °/s: a ratio of 8.0. If that ratio is below about 3, the wheel saturates during an ordinary detumble and the rig cannot demonstrate anything. Fix it by increasing J_w (a heavier or larger-radius rotor), raising $\Omega_{\max}$, or lowering J_b — in that order of practicality.

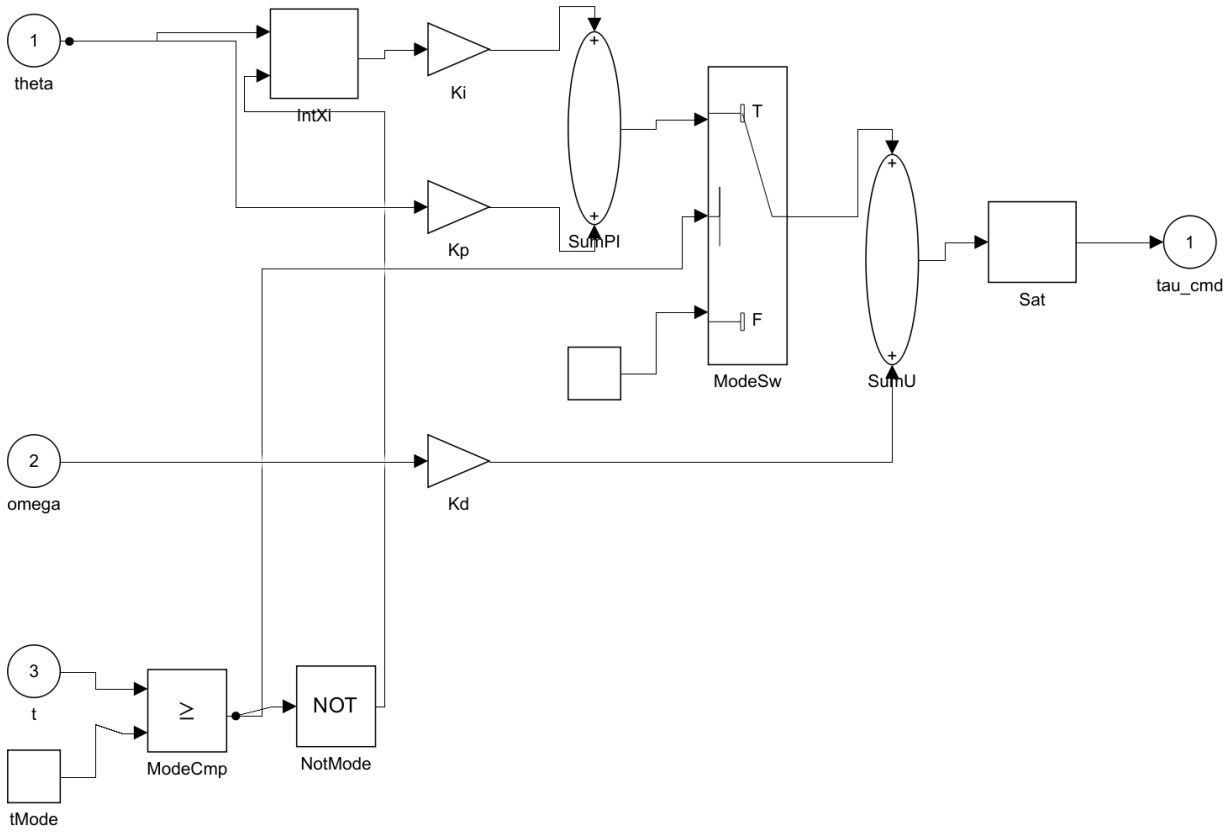


Figure 3: The controller: the PID of Eq. (15). `IntXi` carries the integral state, with its reset port driven by `NOT(mode)` so the integrator is pinned at zero throughout detumble, and its output limited to $\pm\tau_{\max}/K_i$ for anti-windup. `ModeSw` gates the whole proportional-plus-integral branch, leaving rate damping alone to run Mode 1.

14.2 Holding an angle costs wheel speed

In space a reaction wheel holds an attitude at zero steady torque. On a string it cannot, because the filament pulls back. Equilibrium at θ_r needs $\tau_m = -k\theta_r$ forever, and feeding that constant into the wheel equation (5) gives

$$\Omega_{\text{ss}} = \frac{-k\theta_r}{b_w}, \qquad \text{reached with time constant} \quad \tau_w = \frac{J_w}{b_w}. \tag{20}$$

For this rig, holding 45° requires 3.77×10^{-5} N m and settles at 342.7 rad/s — 3273 rpm, or 41 % of saturation — approached over 98 s. Note what that depends on: were b_w zero, Ω_{ss} would be infinite and the wheel would ramp until it saturated. **The only thing stopping runaway is bearing friction**, which is not a property you want to rely on.

Practical consequences: command $\theta_r = 0$ (the string's own rest angle) for long holds, treat large offsets as short manoeuvres, and re-zero the rest angle between sessions.

14.3 The bearing-drag pointing bias

The $+b_w\Omega$ term in (4) is not negligible. It is the source of the steady pointing error derived in Section 10.3: with viscous drag alone, Eq. (14) reduces to $\theta_{\text{ss}} = b_w\Omega/(k + K_p)$, a pointing offset proportional to *stored wheel speed*.

It is worth seeing how exactly that formula holds. An ideal PD detumble from 30 °/s (viscous drag only, true-state feedback) ends with $\Omega = 102.8$ rad/s and $\theta = 0.209°$, and the formula

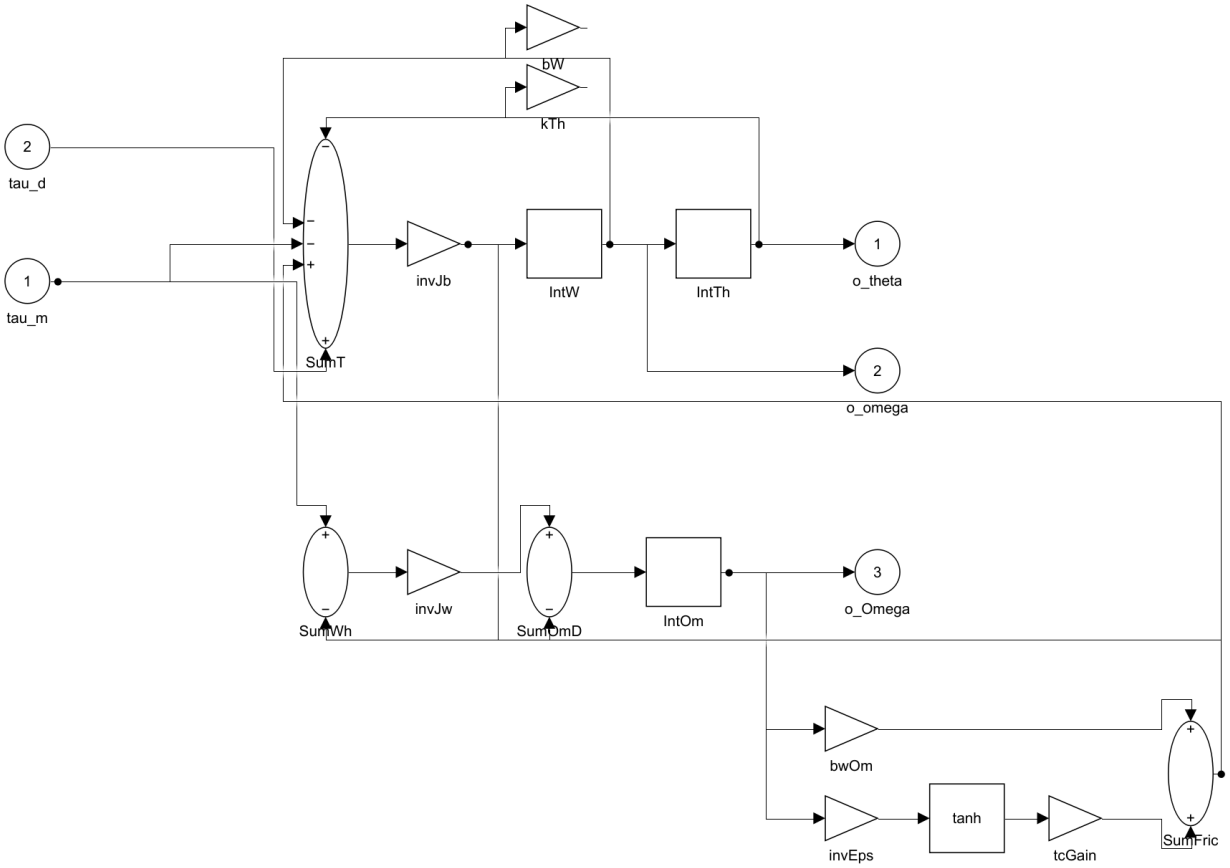


Figure 4: The plant, Eqs. (4)–(5). The two integrator chains are the platform (top) and the wheel (bottom); the `tanh` branch is the Coulomb part of the bearing drag. Note `SumOmD` subtracting $\dot{\omega}$ — that is the term that makes Ω a *relative* rate.

predicts 0.209° — agreement to three decimals. The same check run against the Simulink model of Section 13, this time with Coulomb friction included, gives 0.2656° against a predicted 0.2655°. Two independent implementations and a closed form all landing on the same number is how you know the sign of that term is right.

So a reaction wheel that has absorbed momentum leaves you with a standing pointing error, and that error *grows with how much momentum you are storing*. Section 10.3 removes it with integral action. The other fix is the one used in flight: desaturate the wheel — which on orbit means magnetorquers, and on this bench means briefly gripping the platform.

15 Step 9 — The test campaign

Each run should answer one question and produce one plot that settles it.

Correlating against the model. Log at the IMU rate with timestamps and replay the *recorded* torque command through the simulation. Comparing a simulated run against a hardware run with different inputs proves nothing. Re-fit parameters with the same least-squares you used in Section 5:

```
% data: t (s), w (rad/s) from a wheel-locked release test
th = cumtrapz(t, w);                          % angle by integration
Phi = [-th, -w];                              % Jt*wdot = -k*th - b*w
wd  = gradient(w, t);
```

Run	Procedure	What the plot must show
Free decay	Wheel locked, twist 20°, release, log 7 min.	Clean exponentially decaying sinusoid. Gives k and b. Any beating means a pendulum mode is still live — re-damp and repeat.
Wheel coast-down	Platform clamped, spin up, cut drive, log the encoder.	A single exponential. A knee near zero speed is Coulomb friction; read $p.tc$ off it.
Torque calibration	Platform free, wheel locked to the platform, command a known current step.	$\ddot{\theta}$ proportional to i. The slope is K_t/J_b; it validates $\tau_{\max}$.
Detumble	Hand-spin to ∼30 °/s, release, Mode 1 only.	Rate to zero in about 33 s; wheel below 25 % of saturation; no torque-limit chatter.
Pointing step	From rest, command 30°, hold 60 s.	Settling in about 5 s, overshoot consistent with $\zeta_c = 0.7$, and a visibly climbing wheel speed — that climb is Section 14, not a bug.
Disturbance	At rest in Mode 2, tap the platform.	Recovery to the same angle; residual offset should track Eq. (14).

Table 6: The campaign. Run them in this order — each one supplies a parameter the next one needs.

```
par = Phi \ (p.Jt*wd);
fprintf('refit k = %.4e (was %.4e), b = %.4e (was %.4e)\n', ...
        par(1), p.k, par(2), p.b);
```

Listing 12: Refitting k and b from a recorded free-decay trace

If the refit moves k by more than about 10 %, your gains are built on the wrong plant — regenerate them before the next run.

16 What this rig will not tell you about orbit

Be explicit about the gap when you present results, or someone will over-claim on your behalf.

- **The spring and the damping are not there in space.** Set $k = 0$ and $b = 0$ and re-run every design before you believe it will fly. In particular, the angle observability of Section 9 vanishes entirely, so a flight estimator *must* have an absolute attitude reference — the sun sensors and magnetometer. The testbed's weak observability was a hint; on orbit it is zero.
- **One axis hides the hard part.** $\boldsymbol{\omega} \times \mathbf{J}\boldsymbol{\omega}$ is identically zero here. Three-axis control has coupling, nutation, and a wheel-momentum envelope that is a solid in $\mathbb{R}^3$ rather than an interval.
- **Desaturation is missing.** On the bench you can grab the platform. In orbit you need magnetorquers against the Earth's field, which cannot produce torque about the field direction — a constraint with no bench analogue.
- **Disturbances differ.** Air drag on the bench is viscous and roughly constant; on orbit the dominant secular torques are aerodynamic, gravity-gradient and solar pressure, and they are attitude-dependent.

What *does* transfer, unchanged: the momentum-exchange arithmetic, the saturation logic, the estimator structure, the discrete loop timing, and the habit of checking every gain against $\tau_{\max}$ before trusting it.

A File manifest

File	Contents
`rw1dof_params.m`	All measured constants, SI, one place.
`rw1dof_ode.m`	Nonlinear plant, Eqs. (4)–(5).
`rw1dof_lin.m`	State-space matrices, Eq. (6).
`rw1dof_check.m`	Momentum self-check. Run after every plant edit.
`rw1dof_design.m`	PD and PID gains, with the saturation check.
`rw1dof_est.m`	Augmented model and steady-state estimator gain.
`rw1dof_sim.m`	The realistic discrete loop of Section 12.
`rw1dof_build_slx.m`	Builds `rw1dof.slx` from code, Section 13.
`rw1dof_verify_slx.m`	Checks the Simulink model against `rw1dof_ode.m`.

B Toolboxes, and the no-toolbox path

The controller in this document is PID throughout, and the gains come from the closed-form expressions in Eqs. (13) and (17) — plain arithmetic on measured parameters, no toolbox and no state-space control theory required. That is deliberate: a controller you can derive by hand is one you can debug at the bench.

The Control System Toolbox is needed only for the *analysis* and *estimation* helpers: `ss`, `c2d`, `damp`, `obsv`, `ctrb`, `kalman` and `dlqe`. Everything else — the plant, `ode45`, the discrete loop, the controller itself — is base MATLAB.

Without the toolbox you can still do the whole exercise: the PID gains are already closed-form, a fixed-gain complementary filter replaces `dlqe`, and `expm([A B; 0 0]*Ts)` does the discretisation. The physics, the self-check and the conclusions are unchanged.

A note on what was deliberately left out. There is no LQR or pole-placement design here. For a single axis with one actuator, the PID of Section 10.3 meets every requirement the rig has, its three gains map directly onto quantities you can reason about at the bench (stiffness, damping, bias rejection), and it ports to the flight microcontroller as four lines of C. Optimal state-feedback becomes worth the machinery when there are several actuators to coordinate or genuinely competing objectives to trade off — on three axes with momentum management, for instance. Reach for it then, not now.

C Reproducibility note

Every figure quoted in this document was generated by the listed code with `rng(7)` and the parameter set in Section 7, on MATLAB R2025b. Re-run `rw1dof_check.m` first: if the momentum drift assertion fails, nothing downstream of it is meaningful.